



Diagrama de Máquina de Estado

Herysson R. Figueiredo
herysson.figueiredo@ufn.edu.br



Diagrama de Máquina de Estado

Também conhecido como **Diagrama de Transição de Estados** ou simplesmente **Diagrama de Estados**.

É um diagrama comportamental empregado para descrever como um sistema se comporta quando um evento ocorre, considerando todos os estados, transições e ações possíveis de um objeto.



Diagrama de Máquina de Estado

Representa o estado ou situação na qual um objeto pode se encontrar ao longo da execução dos processos em um sistema.

Mostra como um elemento se comporta por meio de um conjunto de transições de estado, a chamada "**máquina de estados**".



Para que serve?

Representar o ciclo de vida de um objeto

Mostra os estados possíveis de um objeto (ex: um pedido, um usuário, uma ordem de serviço) e como ele transita entre esses estados ao longo do tempo.

Exemplo: Um pedido pode estar nos estados: Criado → Confirmado → Enviado → Entregue → Finalizado.



Para que serve?

Modelar sistemas reativos

É ideal para sistemas que respondem a eventos, como sistemas embarcados, interfaces interativas ou controladores de processos.

Exemplo: Um semáforo inteligente que muda de estado com base em sensores ou tempo.



Para que serve?

Ajudar na detecção de falhas lógicas

Permite visualizar transições inválidas, lacunas de tratamento de eventos, ou estados que não têm saída, tornando mais fácil depurar o sistema.



Para que serve?

Documentar regras de negócio

Facilita a comunicação entre desenvolvedores, analistas e stakeholders, descrevendo regras como:

- O que acontece quando o usuário clica em "Cancelar"?
- Quais eventos levam ao encerramento de uma sessão?



Para que serve?

Complementar outros diagramas UML

É um diagrama comportamental, que complementa diagramas estruturais como:

- Diagrama de Classes (estrutura estática)
- Diagrama de Casos de Uso (interações com atores)
- Diagrama de Atividades (fluxo geral)



Elementos de um diagrama de estados

Nº	Elemento	Nome em Português	Símbolo UML
1	Initial State	Estado Inicial	● (círculo preto sólido)
2	Final State	Estado Final	◎ (círculo com centro)
3	State	Estado Simples	Retângulo arredondado
4	Composite State	Estado Composto	Retângulo com subdivisão
5	Submachine State	Submáquina	Retângulo com "aba"



Elementos de um diagrama de estados

Nº	Elemento	Nome em Português	Símbolo UML
6	Transition	Transição	→ (seta com rótulo)
7	Junction	Junção	● (círculo pequeno)
8	Choice	Ponto de Decisão	◇ (losango)
9	Entry Point	Ponto de Entrada	Círculo com seta →
10	Exit Point	Ponto de Saída	Círculo com um X
11	History State	Estado de Histórico	H ou H* em círculo



Estado (simples)

Condição ou situação na vida de um objeto que satisfaz alguma condição, realiza alguma atividade ou espera um evento.



Estado



Estado inicial / Estado final

Estado inicial: Determina o início da modelagem dos estados de um elemento.



Estado final: Indica o final dos estados modelados para o elemento.





Transição

Movimento de um estado para outro estado. Representa um evento que causa uma mudança no estado de um objeto, levando a um novo estado.





Transição

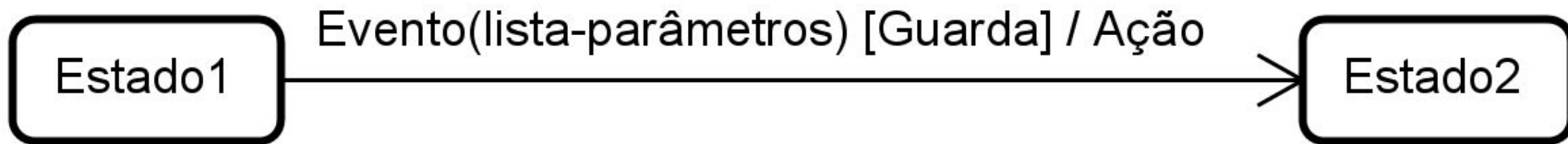
Ocorre da seguinte forma:

- Um elemento está em um estado inicial;
- Um evento ocorre;
- Uma ação é realizada;
- O elemento muda para um estado distinto.



Transição

Exemplo:





Transição

Trigger (Gatilho / Evento)

Um evento que dispara a transição de um estado para outro.

Exemplo: botaoPressionado, cartaoInserido, tempoEsgotado

Sem o trigger, a transição não ocorre.



Transição

Guard (Condição de Guarda)

Uma condição **de valor lógico** que **deve ser verdadeira** para que a transição ocorra após o trigger.

É opcional. Se estiver presente e for falsa, a transição não é executada, mesmo que o evento ocorra.



Transição

Action (Ação ou Efeito)

A atividade executada assim que a transição ocorre (ou seja, depois que o trigger é ativado e a guard é verdadeira).

Pode envolver: chamadas de métodos, mensagens, logs, etc.



Transição

Exemplos:

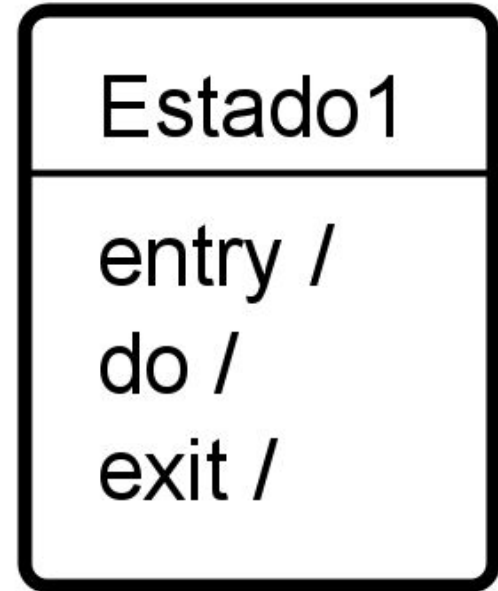
- clicarBotao [usuarioLogado] / abrirMenu()
- loginEnviado [senhaCorreta] / abrirPainel()





Estado - Comportamento interno

O comportamento interno de um estado representa ações que ocorrem enquanto o objeto está naquele estado específico, independentemente de transições externas.





Comportamentos de Estado: Entry / Do / Exit.

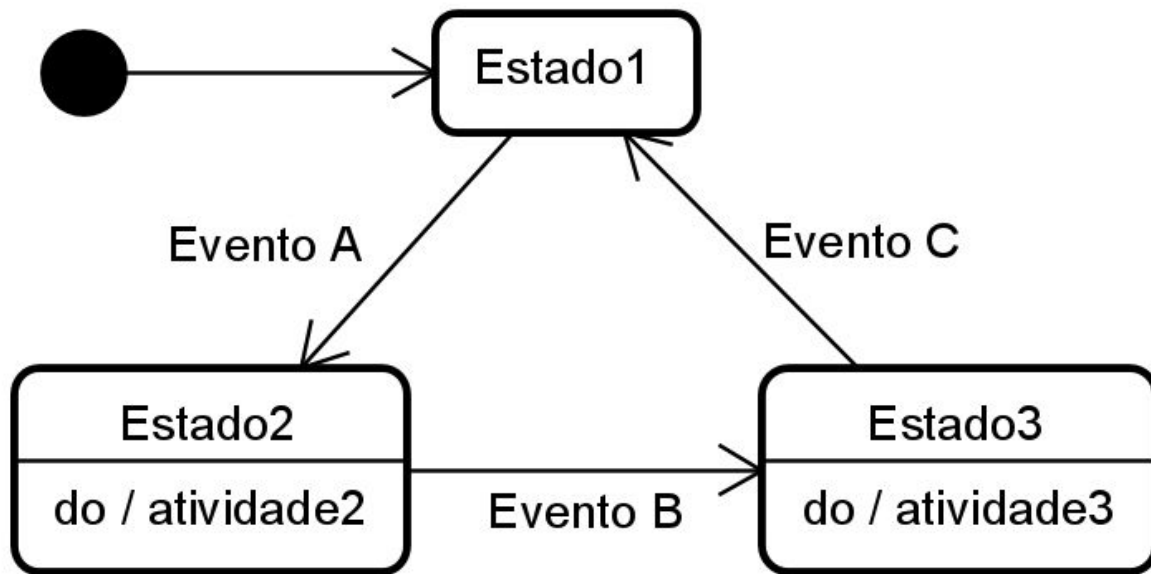
Comando	Quando é executado?	Exemplo
entry	Ao entrar no estado	entry / exibirMensagemBoasVindas()
do	Enquanto o estado está ativo	do / monitorarSessao()
exit	Ao sair do estado	exit / limparVariaveis()



Comportamentos de Estado: Entry / Do / Exit.

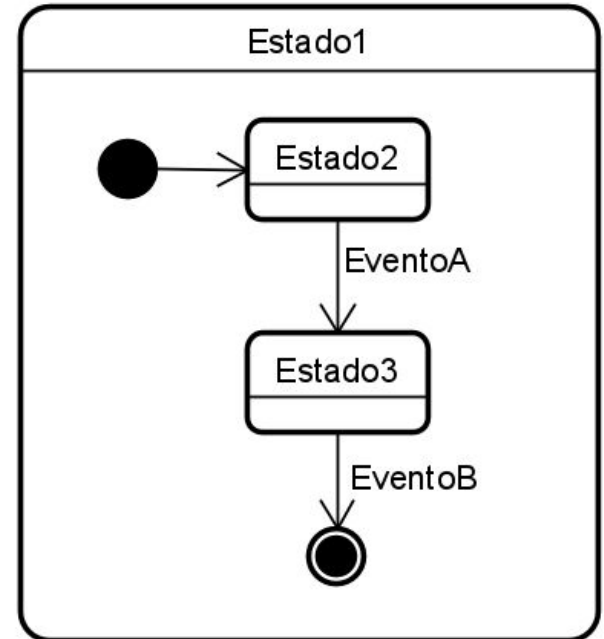
Logado
entry / carregarDadosUsuario() do / aguardarInteracao() exit / salvarSessao()

Exemplo



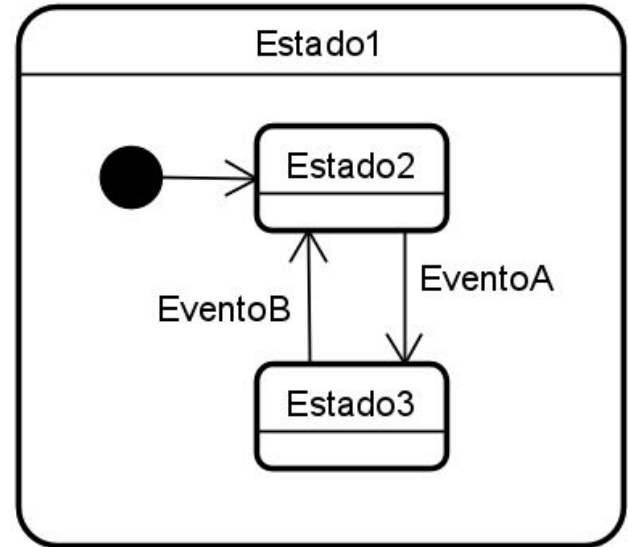
Estado composto:

Estado que possui sub-estado.



Estado composto:

Estado que possui sub-estado.



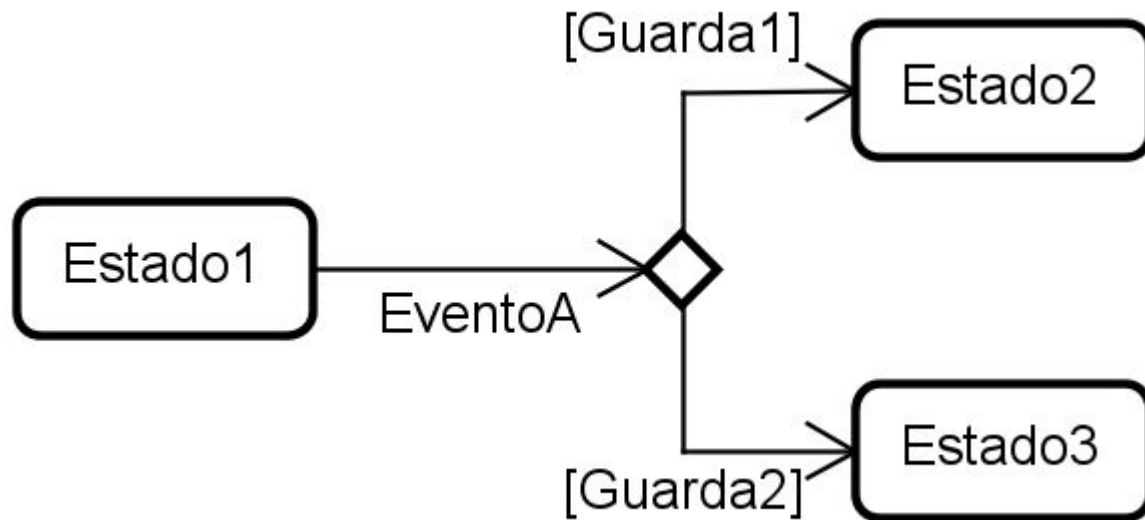


Decisão

Permite ramificações com múltiplas saídas, onde a condição para seguir um determinado caminho é avaliada em tempo de execução, após a transição de entrada.



Decisão



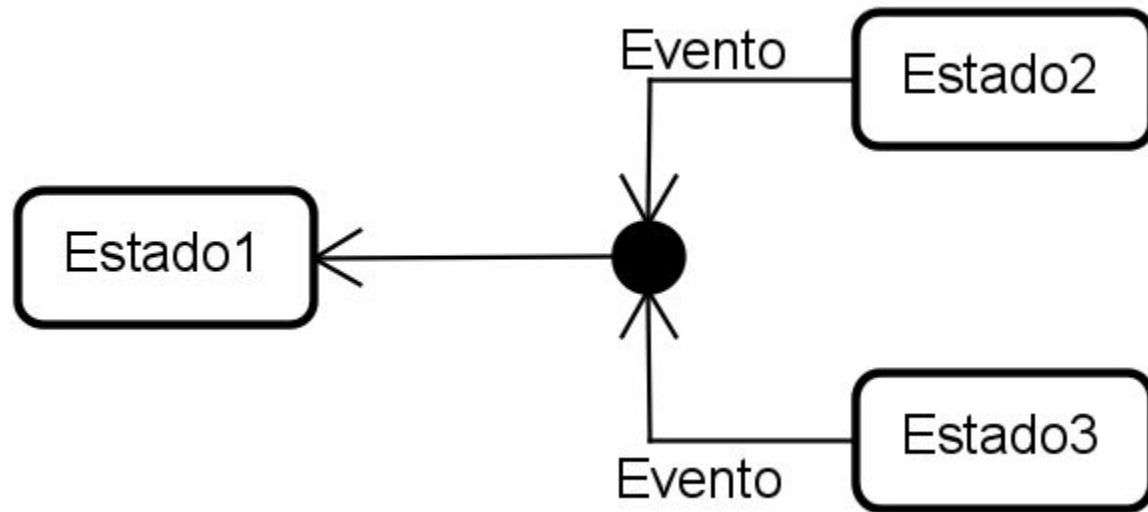


Junção

Usado para conectar transições em um diagrama de máquina de estados. Ele serve para dividir ou fundir caminhos de forma técnica, sem representar um estado real e sem lógica condicional explícita como no choice.



Junção





Decisão / Junção

Aspecto	choice (◆)	junction (●)
Símbolo	Losango	Círculo preto
Tipo de decisão	Dinâmica (condição avaliada na hora)	Estática (encadeia transições)
Guarda	Obrigatória	Opcional
Ação associada	Não	Não
Exemplo típico	Decisão com múltiplos caminhos	Convergência de caminhos



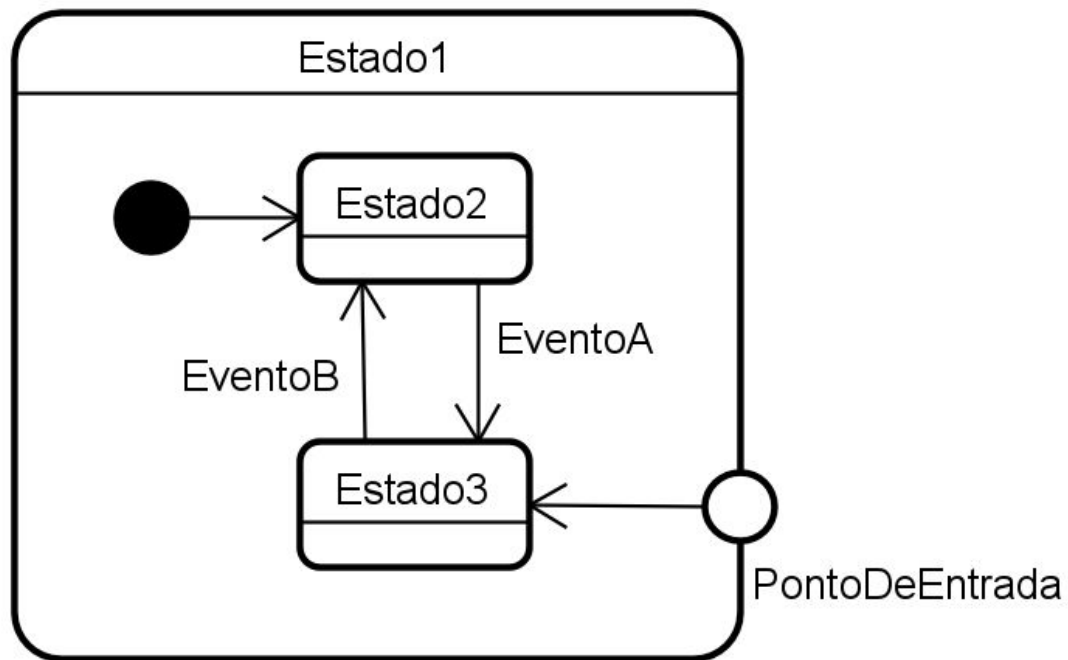
Entry Point (Ponto de Entrada)

Representa um ponto específico de entrada em um estado composto ou submáquina.

Permite entrar em um subestado específico, ignorando o estado inicial padrão.

Usado quando a transição não deve iniciar pelo subestado padrão, mas sim por um ponto alternativo.

Entry Point (Ponto de Entrada)





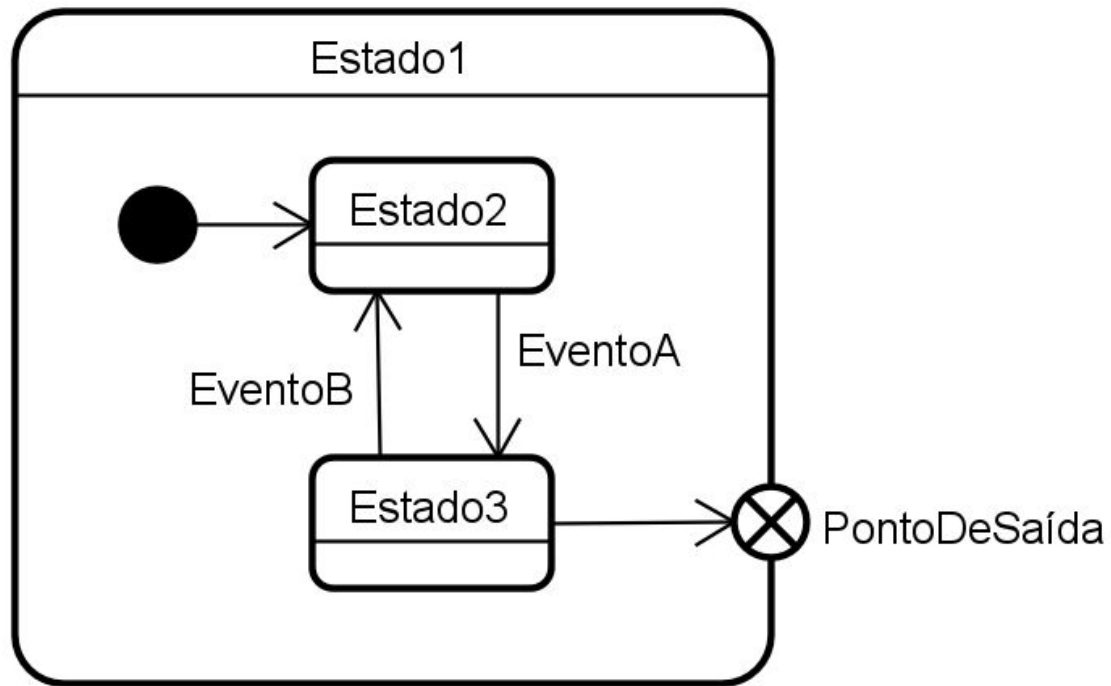
Exit Point (Ponto de Saída)

Representa um ponto específico de saída de um estado composto ou submáquina.

Usado para sair imediatamente de um estado composto, com uma transição direcionada e controlada.

Permite representar saídas antecipadas, como cancelamentos, erros ou exceções.

Exit Point (Ponto de Saída)





Entry Point x Exit Point

Situação	Solução com Entry/Exit
Reentrar em um estado composto em um ponto específico	Entry Point
Sair de um estado composto por um evento específico	Exit Point
Controlar como e por onde o fluxo entra e sai de submáquinas	Ambos

Submachine State

É um estado que representa outra máquina de estados inteira, ou seja, um diagrama de estados encapsulado dentro de um único estado em outro diagrama.





Estado de histórico

O estado de histórico representa a última subposição ativa dentro de um estado composto, permitindo que o sistema retorne ao último subestado ativo em vez de reiniciar do início.





Estado de histórico

Imagine um sistema que:

- Estava em um estado composto com vários subestados.
- Foi interrompido (por pausa, erro, evento externo).
- Precisa retornar ao subestado exato onde estava antes da interrupção.

O estado de histórico evita recomeçar do início do estado composto.



Estado de histórico

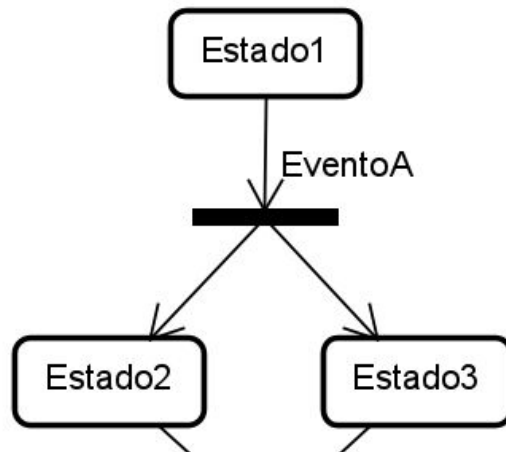
Tipo	Símbolo UML	Significado
H	H em um círculo	Histórico simples : retorna ao último subestado visitado, sem profundidade
H*	H* em um círculo	Histórico profundo : retorna ao último subestado visitado recursivamente , incluindo subníveis

Fork (Divisão)

O fork representa a divisão de uma transição em vários fluxos simultâneos ($1 \rightarrow N$).

Uso:

- Um único estado se divide em várias regiões paralelas.
- Todos os destinos são ativados ao mesmo tempo.

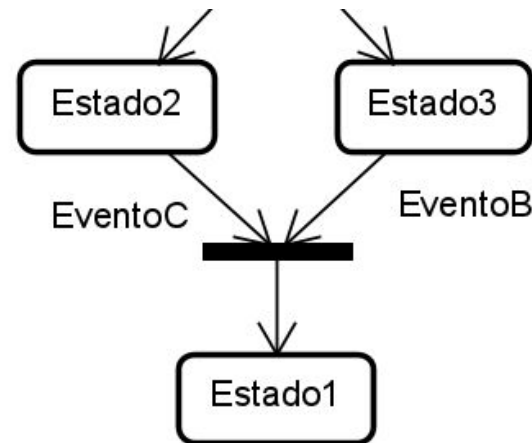


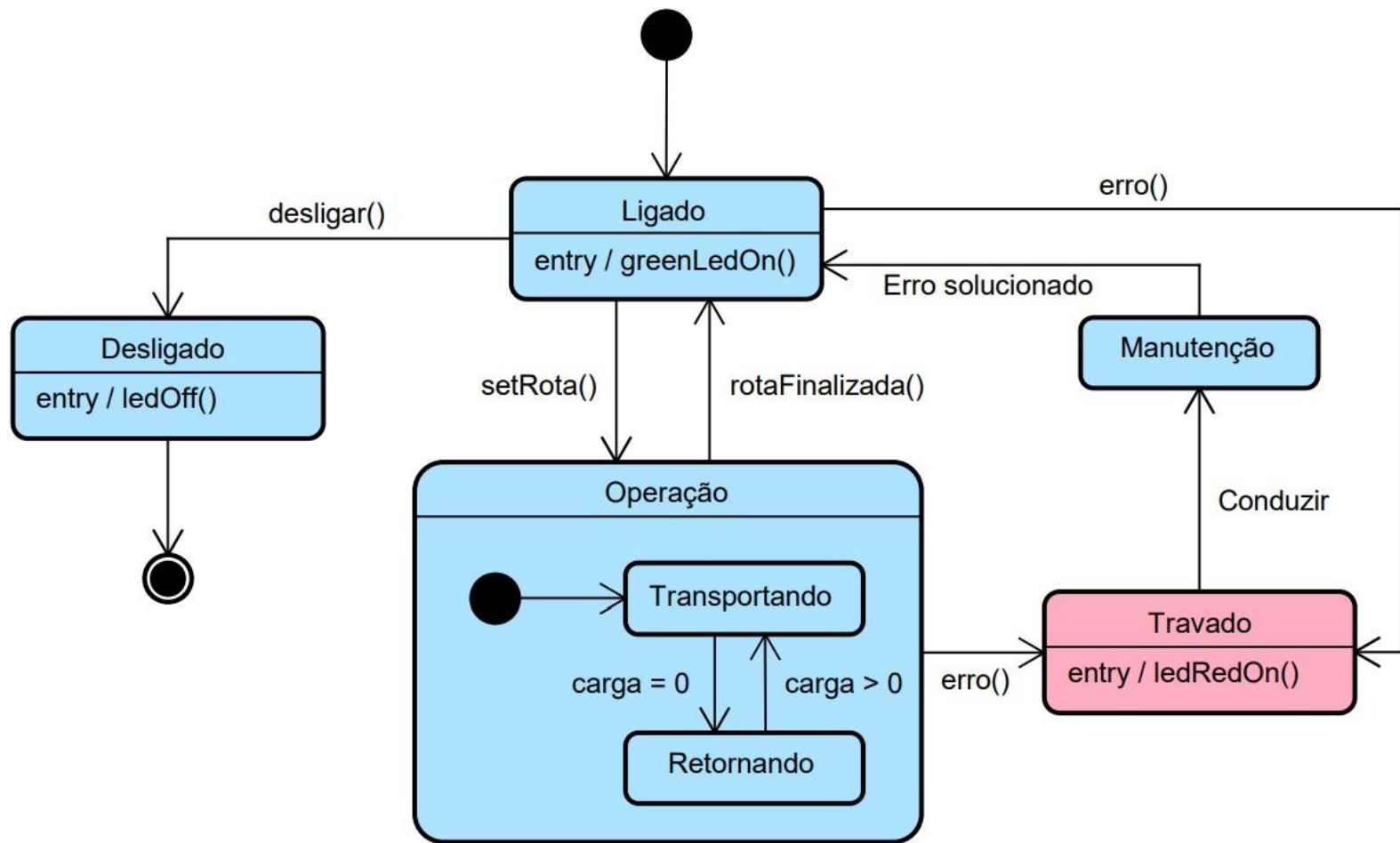
Join (Junção)

O join representa a sincronização de múltiplas transições concorrentes em um único estado ($N \rightarrow 1$).

Uso:

- Espera todas as regiões paralelas concluírem suas execuções antes de continuar.
- O fluxo só segue após todas as entradas estarem completas.







Como criar um diagrama de estados

- Determinando o estado inicial e o estado final
- Identificando todos os estados possíveis para o processo modelado
- Use setas ou linhas para destacar as transições de controle de um estado para outro, conectando origem e destino
- Rotule os eventos que disparam essas transições



Como criar um diagrama de estados

- Estabeleça condições de guarda para assegurar que as transições são apropriadas e relevantes. Uma condição de guarda força a verificação da transição contra uma condição antes de prosseguir.



Atividade Prática

Modele o ciclo de vida de um pedido realizado em um restaurante. Estados sugeridos:

- Pedido Criado
- Pedido Confirmado
- Em Preparo
- Pronto
- Entregue
- Cancelado

Desafio extra: Adicione guards (ex: `[clienteDesistiu]`) e ações (ex: `/notificarCozinha()`).



Atividade Prática

Uma máquina vende lanches ou bebidas.

- Crie estados como: Ociosa, Esperando Seleção, Verificando Estoque, Liberando Produto, Encerrando Transação.
- Use triggers como: inserirMoeda, selecionarProduto.
- Adicione um estado composto para "Preparando Produto"

Atividade Prática

Criar um diagrama de estados para o personagem principal de um jogo.





Atividade Prática

Imagine que você está modelando o comportamento de um sinistro de veículo no sistema da seguradora. Um sinistro é o evento em que o carro segurado sofre um acidente, furto ou outro dano, e o segurado aciona a cobertura.

Seu objetivo é modelar os possíveis ESTADOS do sinistro, os EVENTOS (triggers) que causam transições entre estados, as CONDIÇÕES (guards) e as AÇÕES executadas durante essas transições ou dentro dos próprios estados (entry/do/exit).



Atividade Prática

- Ciclo de vida de um livro no sistema Livir.
- Livros danificados.
- Livro em oferta.



Bibliografia

BEZERRA, Eduardo. Princípios de análise e projeto de sistemas com UML. 2. ed. rev. atual. Rio de Janeiro, RJ: Campus, 2007. 369 p.

WAZLAWICK, Raul Sidnei. Análise e projeto de sistemas de informação orientados a objetos. 2. ed. rev. e atual. Rio de Janeiro: Elsevier, 2011. 330 p. (Série SBC, Sociedade Brasileira de computação) ISBN 978-85-352-3916-4

SOMMERVILLE, Ian. Engenharia de software, 10ª ed., 2019. (Biblioteca Digital)

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML: guia do usuário. 2. ed. Rio de Janeiro, RJ: Ed. Campus, 2006. 474 p.

LARMAN, Craig. Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo. 3. ed., reimpr. Porto Alegre: Bookman, 2007. 695 p. ISBN 978-85-60031-52-8

SOFTWARE GUIDES. **State Machine Diagrams - UML 2 Tutorial**. UML-Diagrams.org, [20--]. Disponível em: <https://www.uml-diagrams.org/state-machine-diagrams.html>.